National University of Computer & Emerging Sciences

Lecture 02

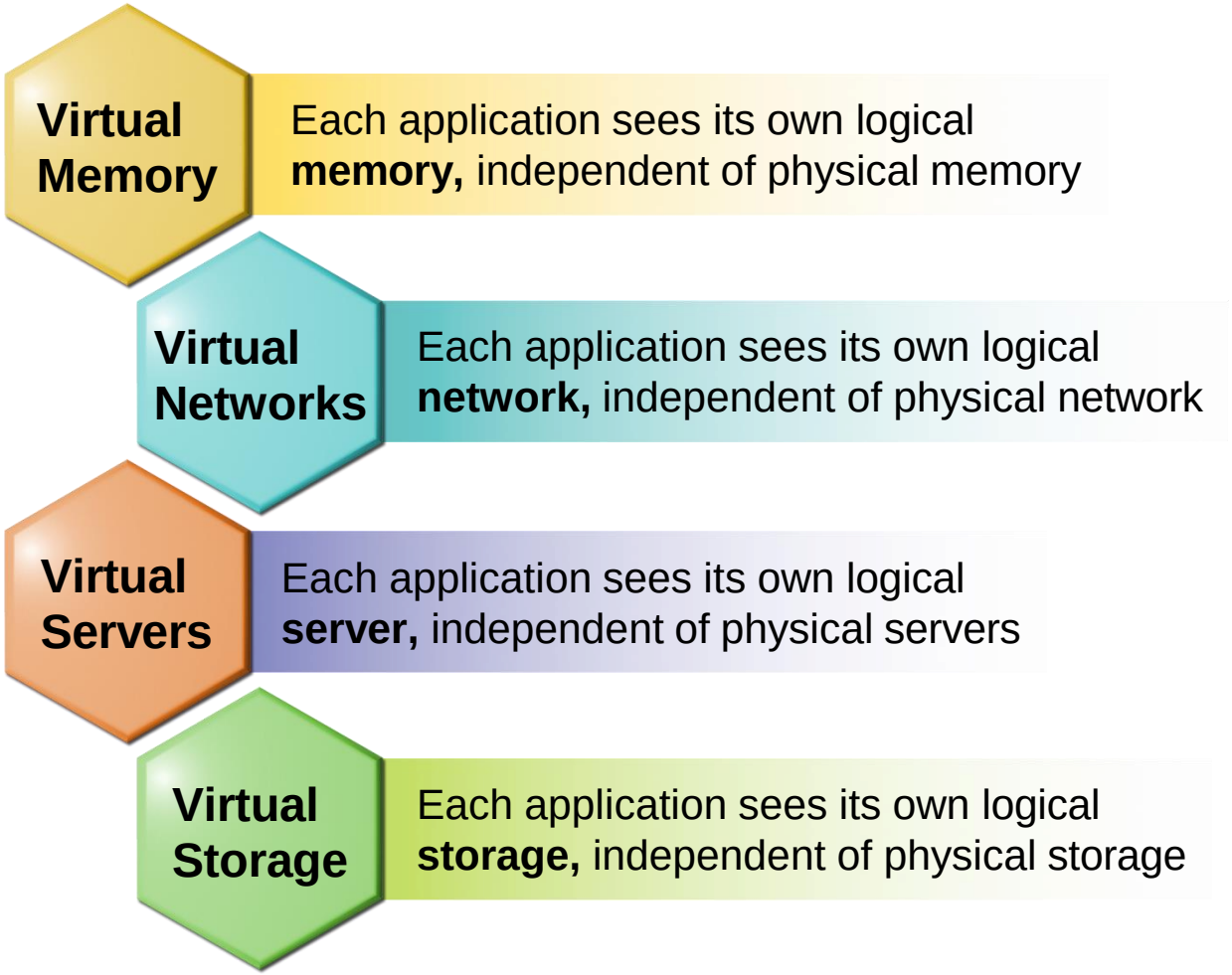
Docker Containers

Virtualization

- From a computing perspective, you might have already done some virtualization if you've ever partitioned a hard disk drive into more than one "virtual" drive.

- What other virtualized resources you have already used?

Virtualization Comes in Many Forms



Virtual Machines

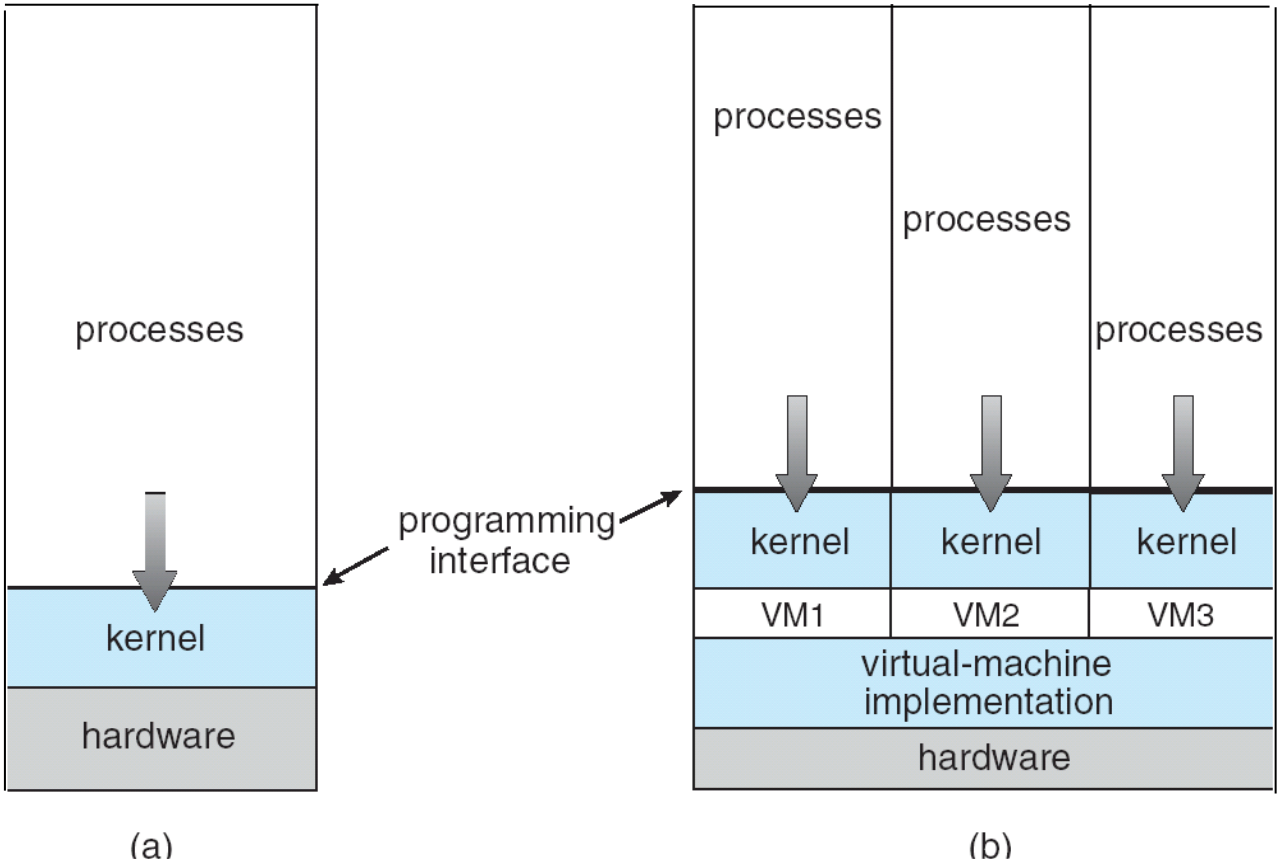
Question

- Can a “small” operating system simulate the hardware of some machine so that
  - Another operating system can run in that simulated hardware?
  - More than one instance of that operating system run on the same hardware at the same time?
  - More than one *different* operating system can share the same hardware at the same time?
- Answer: *Yes*

Solution - *Virtual Machine*

- A virtual machine provides interface *identical* to underlying bare hardware
  - I.e., all devices, interrupts, memory, page tables, etc.
- Virtual Machine Operating System creates illusion of multiple processors
  - Each capable of executing independently
  - No sharing, except via network protocols

Virtual Machines (Cont)



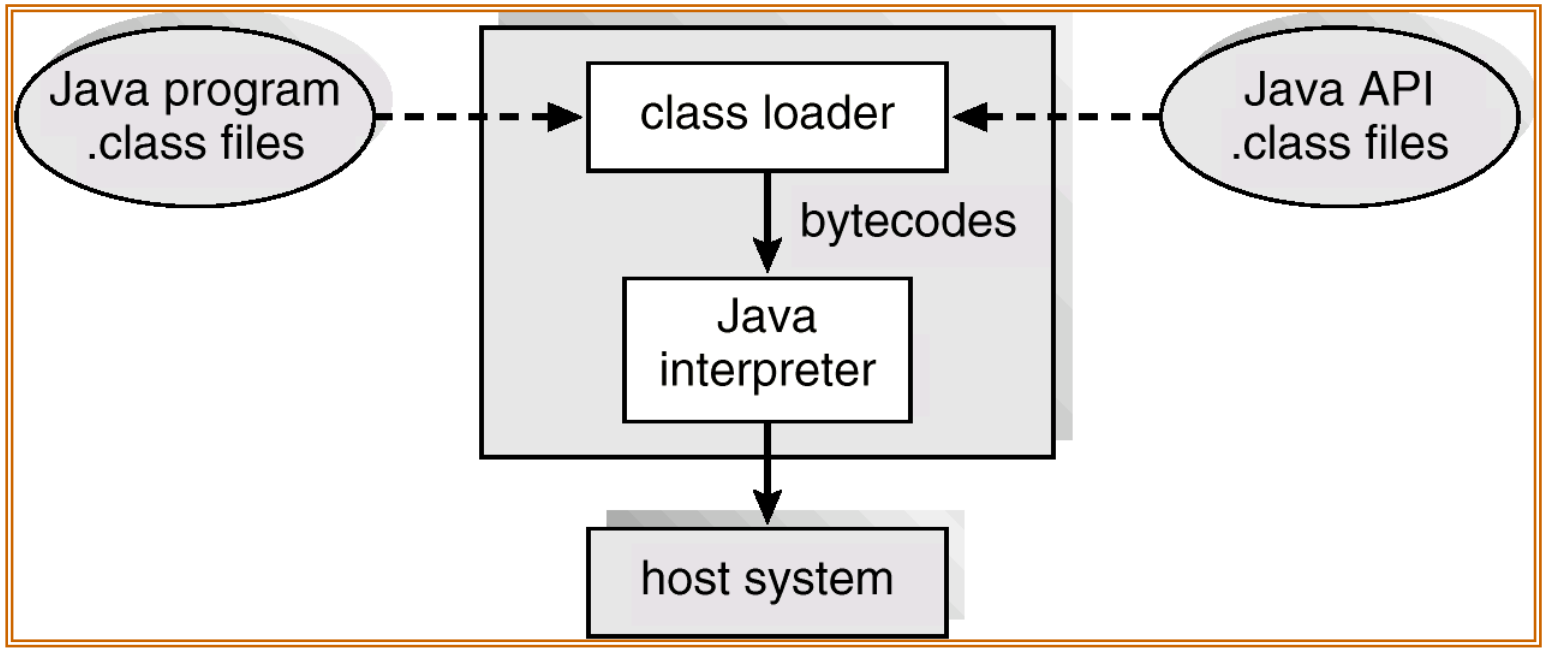
Java Virtual Machine

- Compiled Java programs are platform-neutral byte-codes executed by a Java Virtual Machine (JVM)

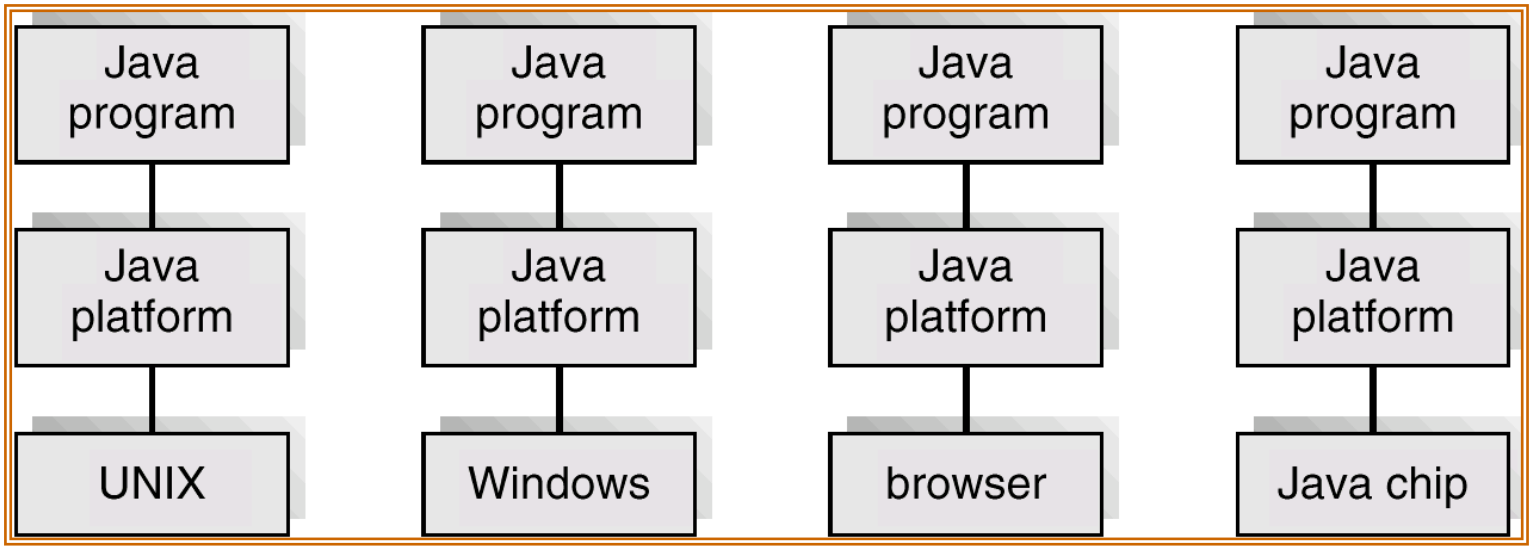
- JVM consists of
  - Class loader
  - Class verifier
  - Runtime interpreter

- Just-In-Time (JIT) compilers increase performance

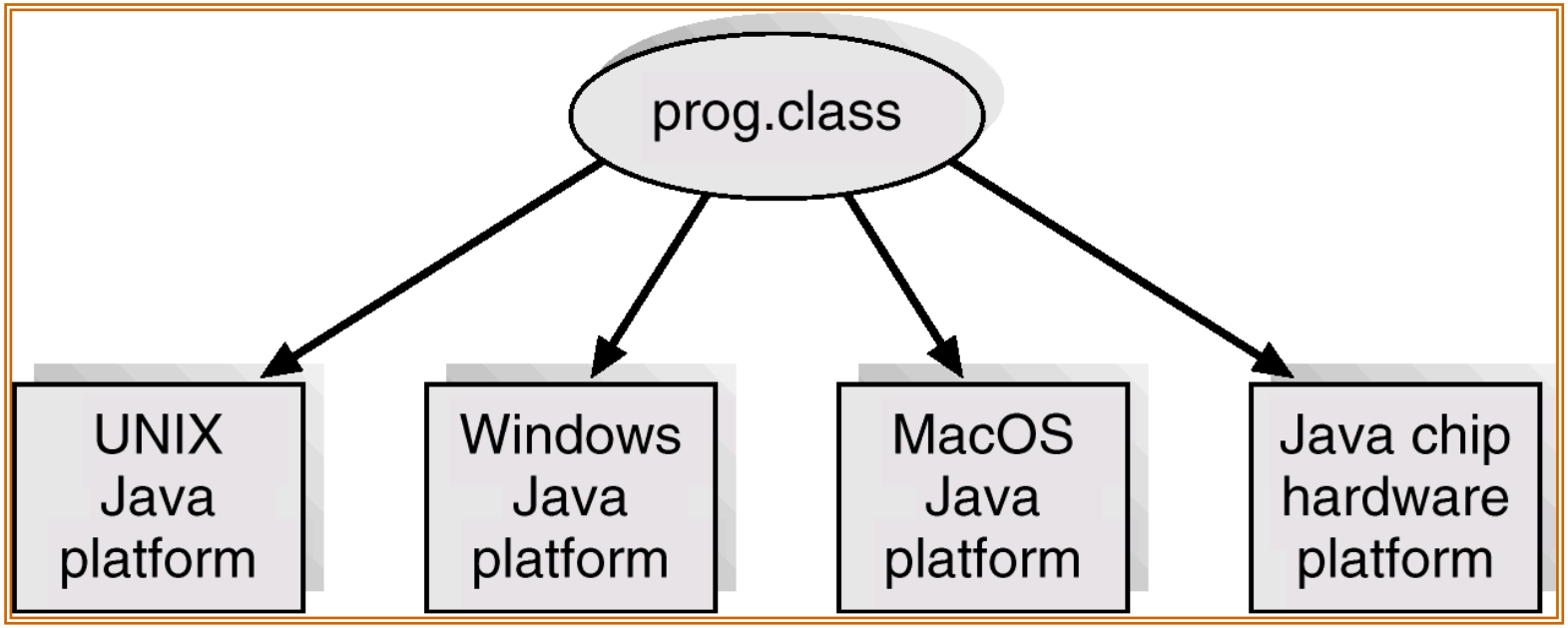
The Java Virtual Machine



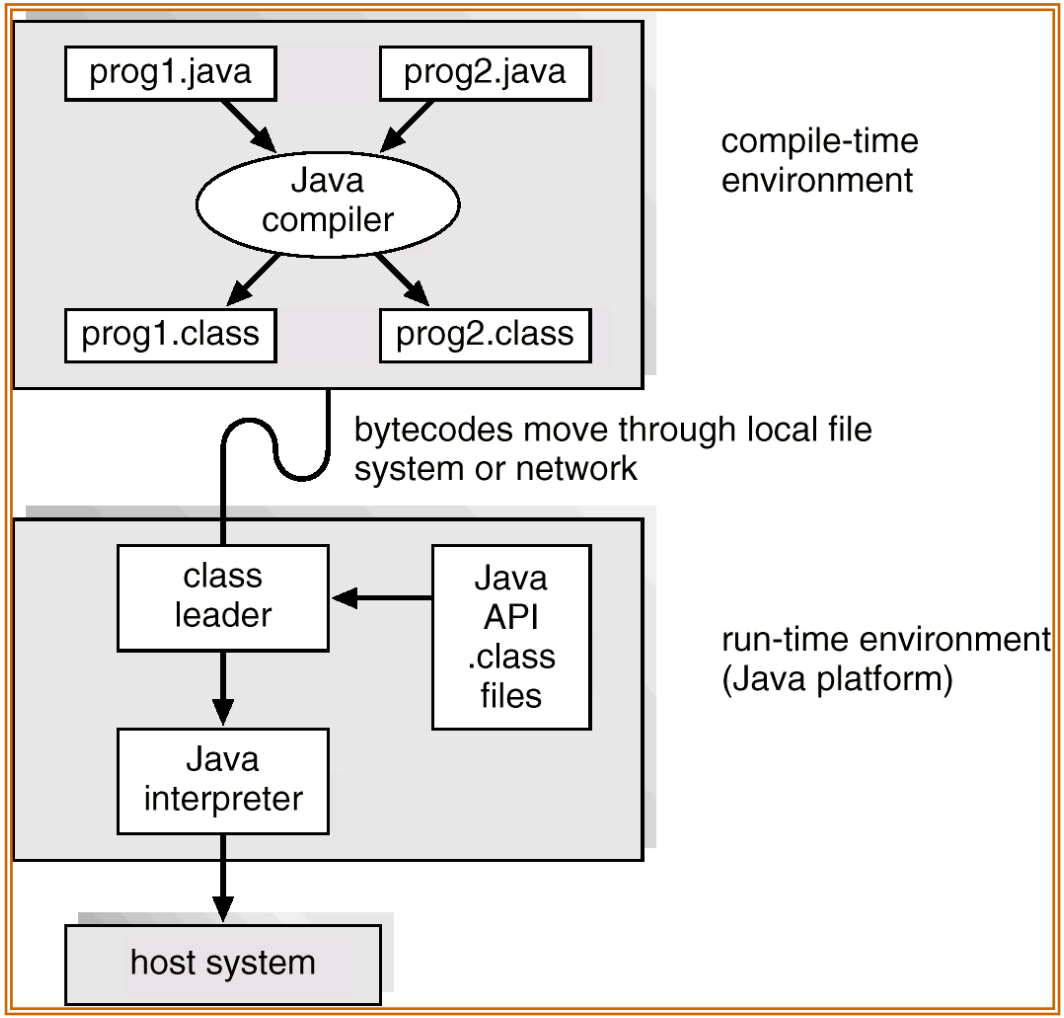
The Java Platform



Java .class File on Cross Platforms

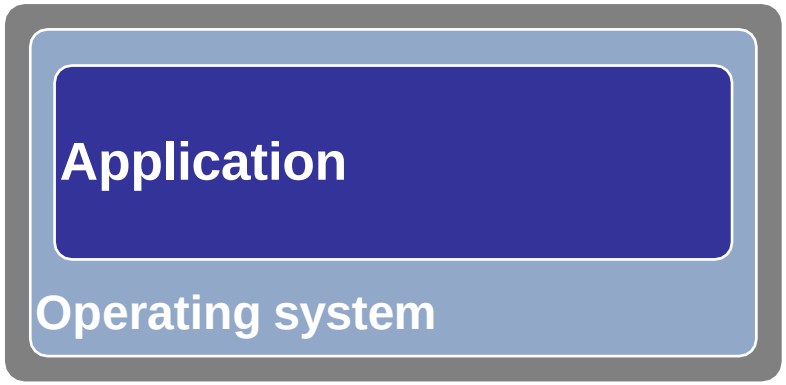


Java Development Environment



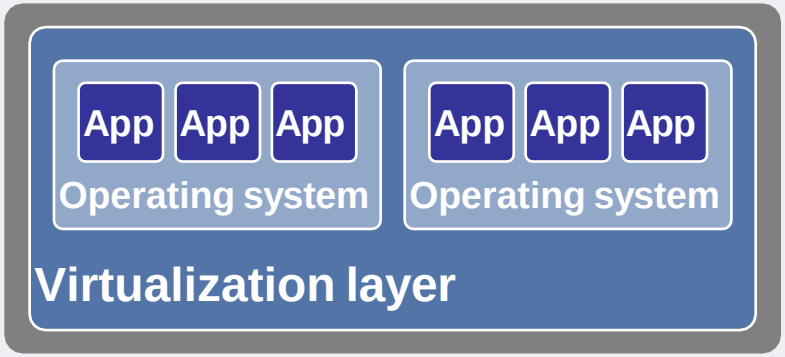
Server Virtualization

Before Server Virtualization:



- Single operating system image per machine
- Software and hardware tightly coupled
- Running multiple applications on same machine often creates conflict
- Underutilized resources

After Server Virtualization:

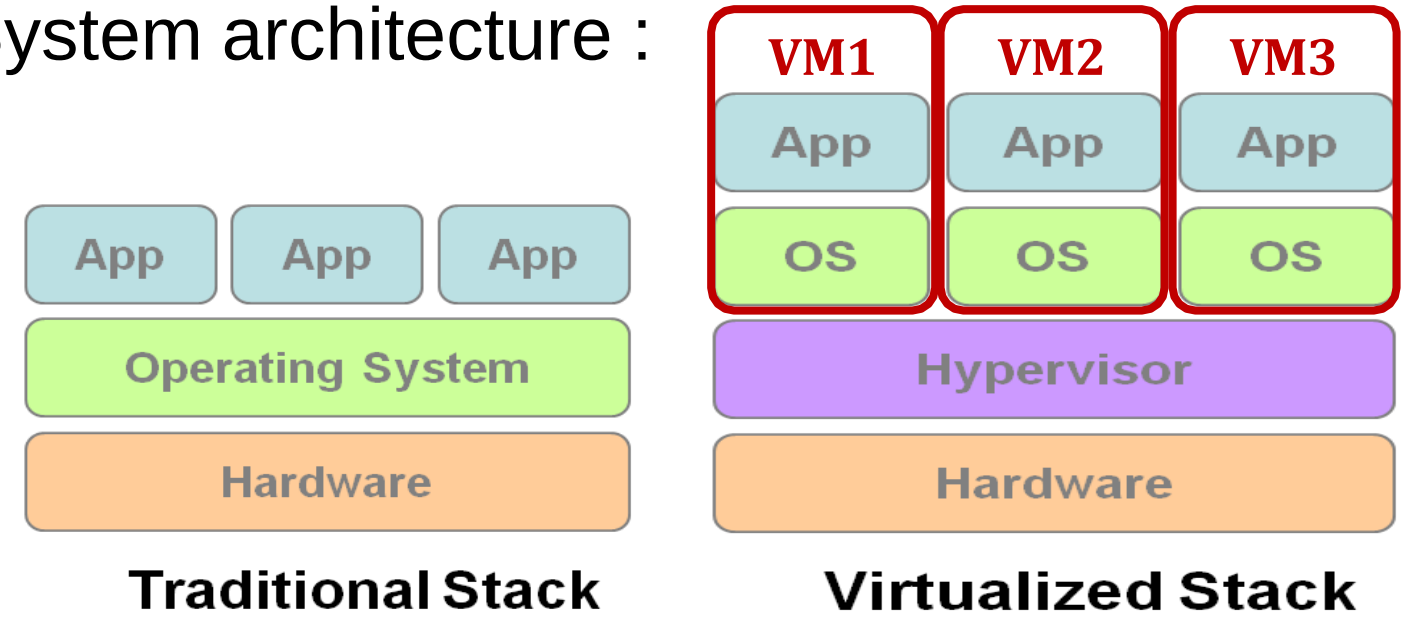


- Virtual Machines (VMs) break dependencies between operating system and hardware
- Manage operating system and application as single unit by encapsulating them into VMs
- Strong fault and security isolation

Virtual Machine Monitor

- What's Virtual Machine Monitor (VMM) ?
  - **VMM** or **Hypervisor** is the software layer providing the virtualization.

- System architecture :



Virtual machines limitations

- VMs take up a lot of system resources.
- Each VM runs not just a full copy of an operating system, but a virtual copy of all the hardware that the operating system needs to run.

- This quickly adds up to a lot of RAM and CPU cycles.

Number of VMs per Host?

- How much is too much?
- 500 VMs on one server host is possible but sometimes less is more. Risk, utilization rates and memory factor into the decision.

- Virtualization doesn't just consolidate as many servers as possible -- it has to actually do something.

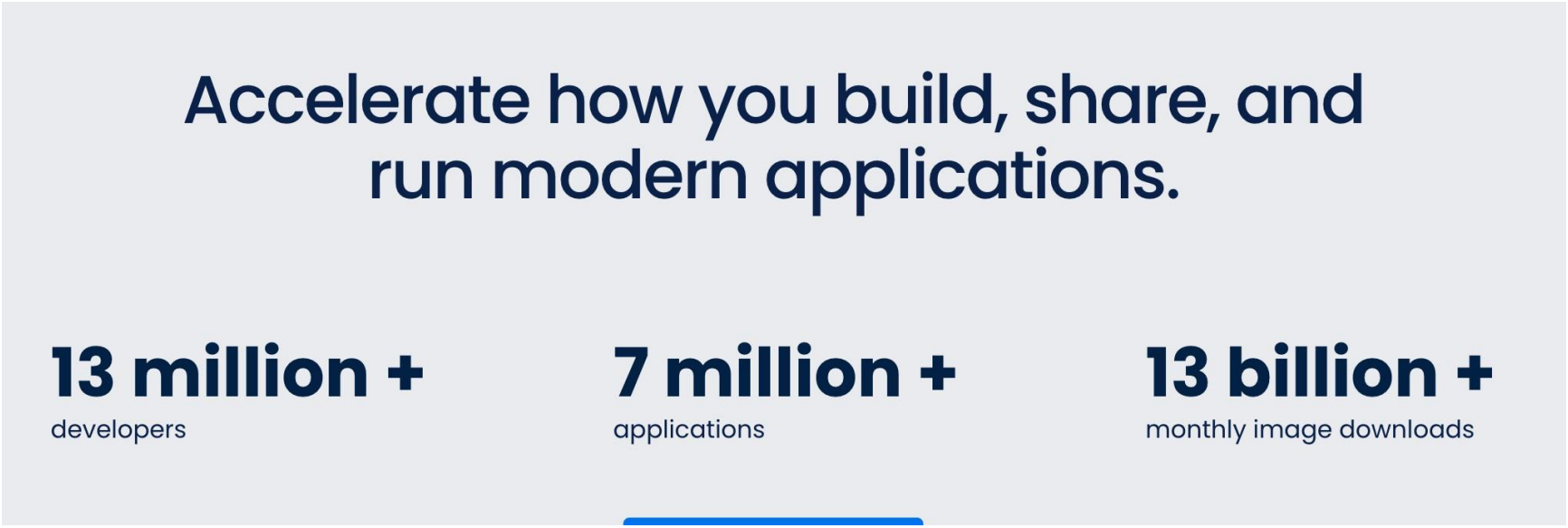
Do we have some alternative?



*Docker containers wrap up **a piece of software in a complete filesystem** that contains everything it needs to run:*
code,
runtime,
system tools,
system libraries
– anything you can install on a server.

This guarantees that it will always run the same, regardless of the environment it is running in

Docker on the rise

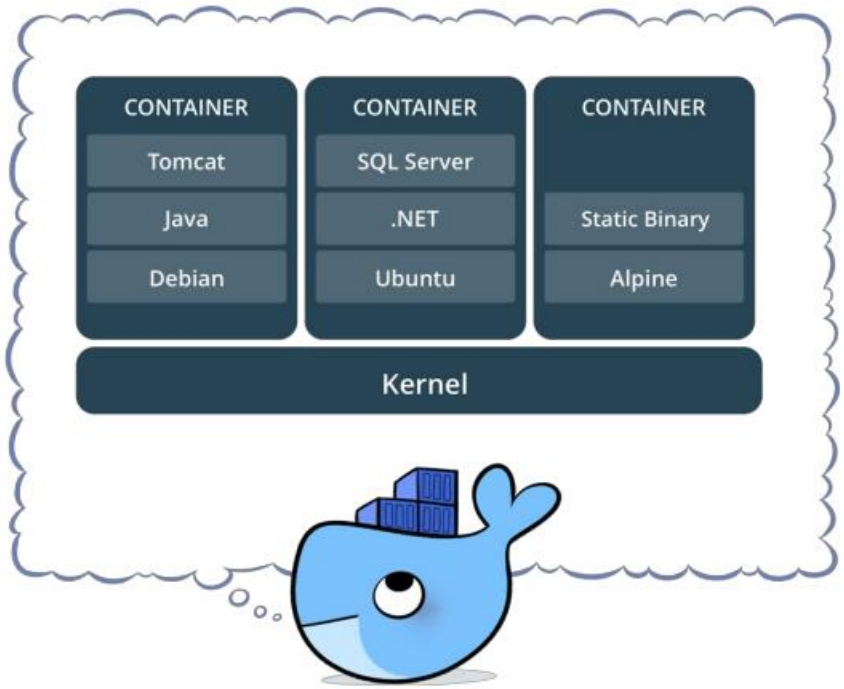


What is Docker?

- A tool that can package an application and its dependencies in a *virtual container*
- Implementation of a container which is portable using a concept of *image*
- Docker uses the host OS kernel, there is no custom or additional kernel inside running containers
- Docker uses resource isolation features of the Linux kernel such as cgroups and kernel namespaces to allow independent “containers” to run within a single Linux instance, avoiding the overhead of starting virtual machines



What is Docker?



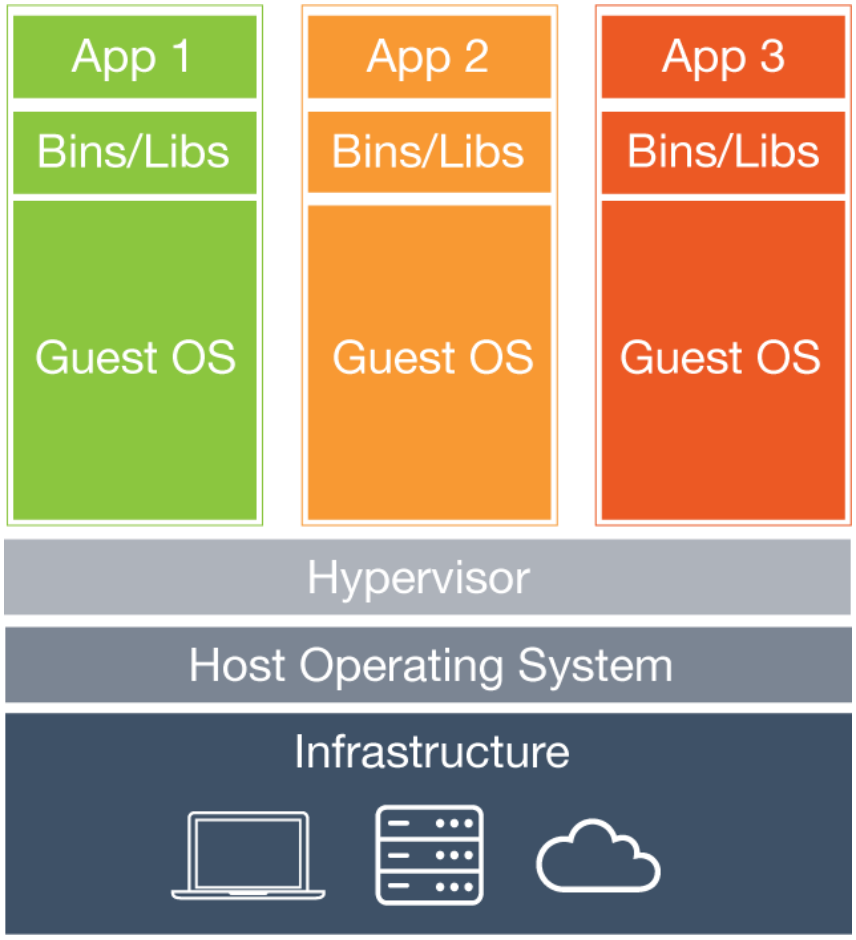
- Standardized packaging for software and dependencies
- Isolate apps from each other
- Share the same OS kernel
- Works for all major Linux distributions
- Available for Windows (Server, since 2016) and MacOS

Docker containers

- Containers running on a single machine all share the same operating system kernel so they start instantly and make more efficient use of RAM.

- You can thus stuff more containers on a server host than VMs.

How is this different?

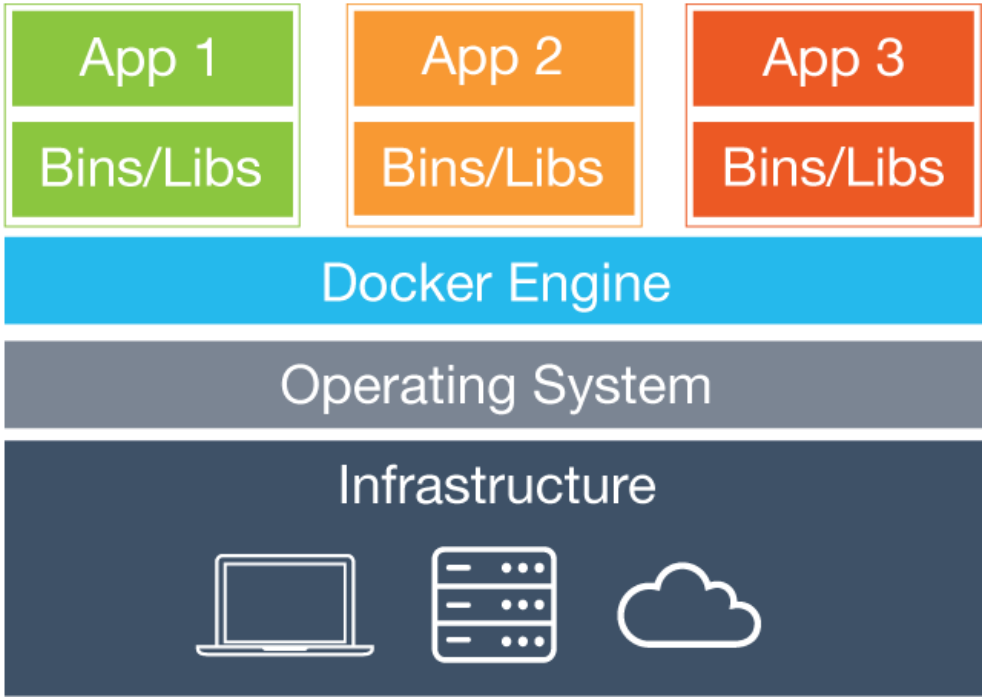


Each virtual machine includes the application, the necessary

- binaries
- libraries
- an entire guest operating system

- all of which may be tens of GBs in size.

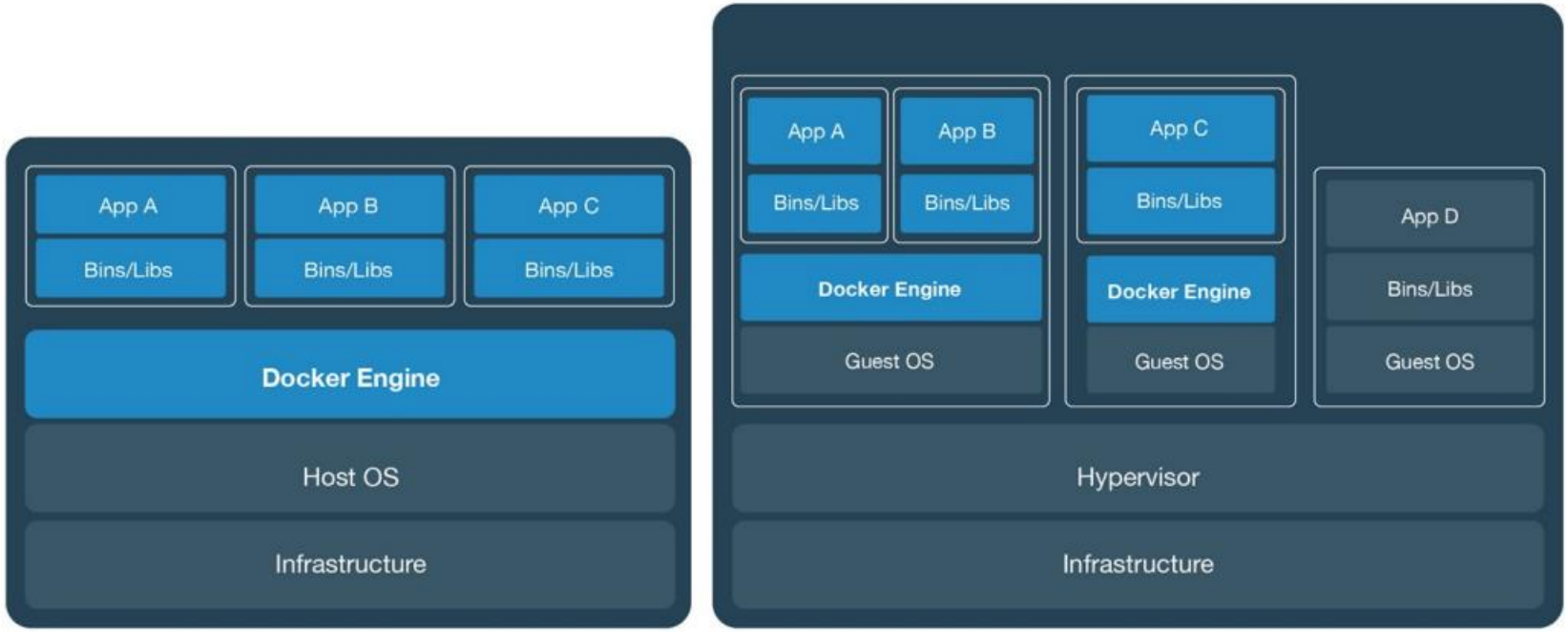
How is this different?



- Containers include the
- application
- all of its dependencies,
- share the kernel with other containers.

- They run as an isolated process in userspace on the host operating system.

Containers versus VMs



VMs vs Docker - Differences

Virtual Machines	Docker
Each VM runs its own OS	All containers share the same kernel of the host
Boot up time is in minutes	Containers instantiate in seconds
VMs snapshots are used sparingly	Images are built incrementally on top of another like layers. Lots of images/snapshots
Not effective diffs. Not version controlled	Images can be diffed and can be version controlled. Dockerhub is like GITHUB
Cannot run more than couple of VMs on an average laptop	Can run many Docker containers in a laptop.
Only one VM can be started from one set of VMX and VMDK files	Multiple Docker containers can be started from one Docker image

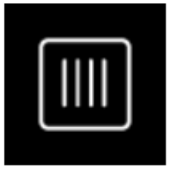
Some Docker vocabulary

Containers	Images
How you **run** your application	How you **store** your application



Docker Image

The basis of a Docker container. Represents a full application
Specified via *Dockerfiles*



Docker Container

The standard unit in which the application service resides and executes



Docker Engine

Creates, ships and runs Docker containers deployable on a physical or virtual, host locally, in a datacenter or cloud service provider

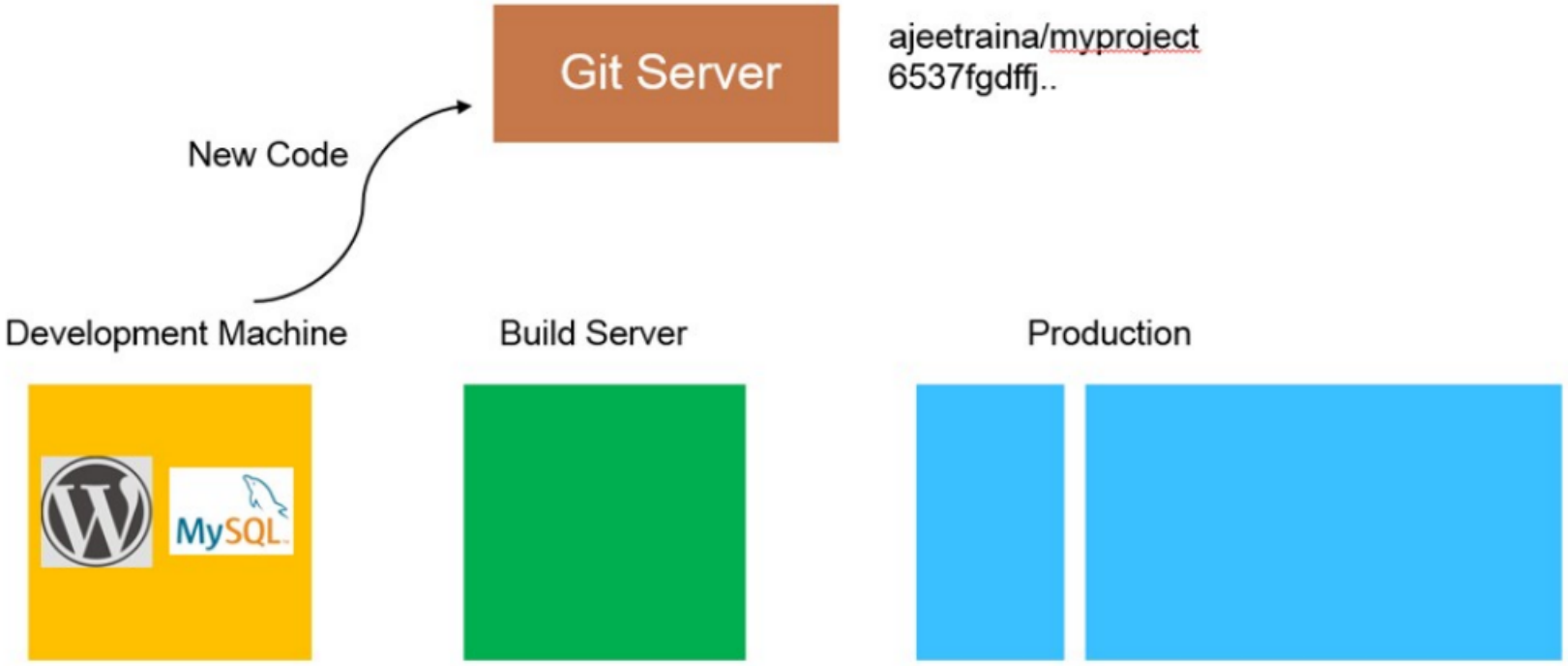


Registry Service (Docker Hub or Docker Trusted Registry)

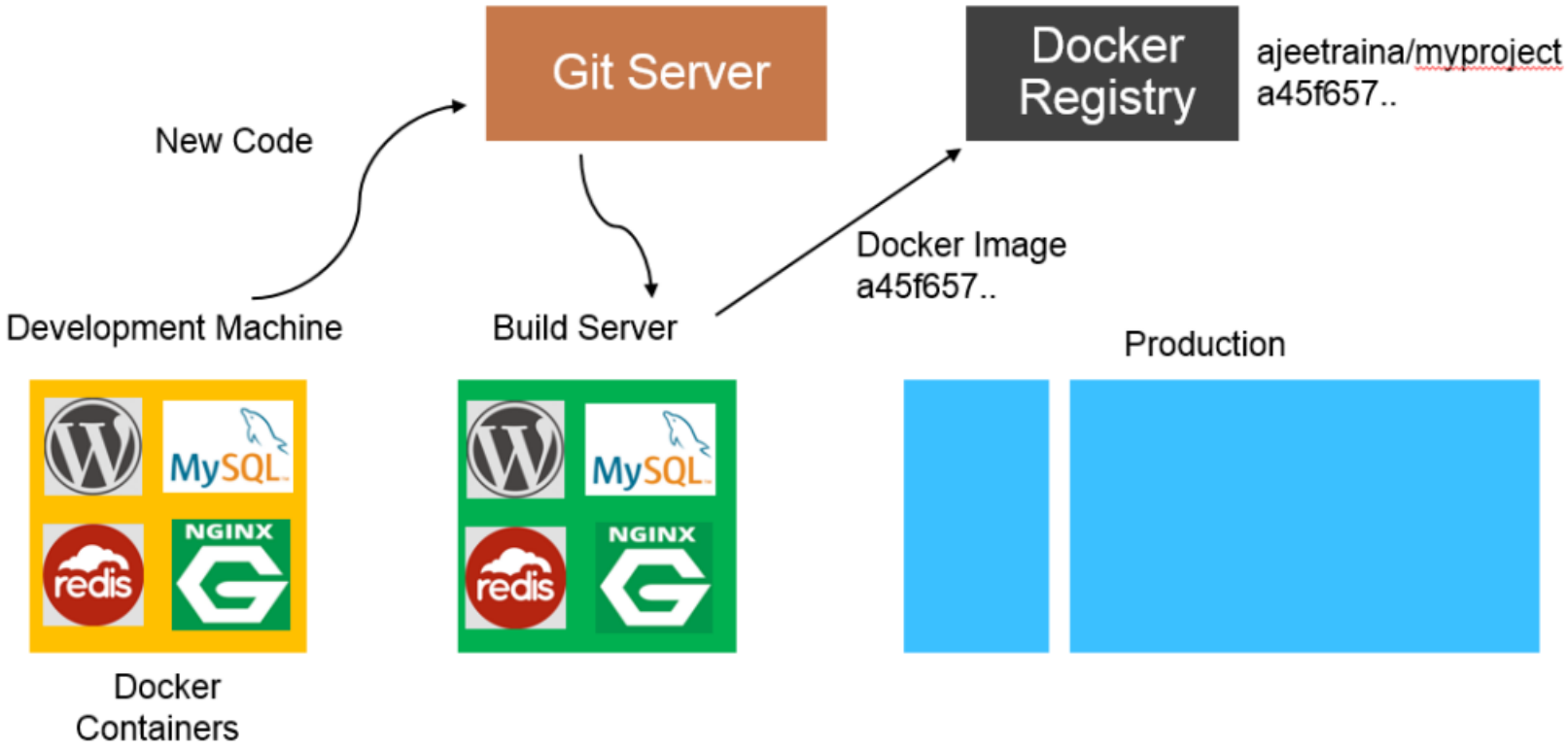
Cloud or server based storage and distribution service for your images

docker

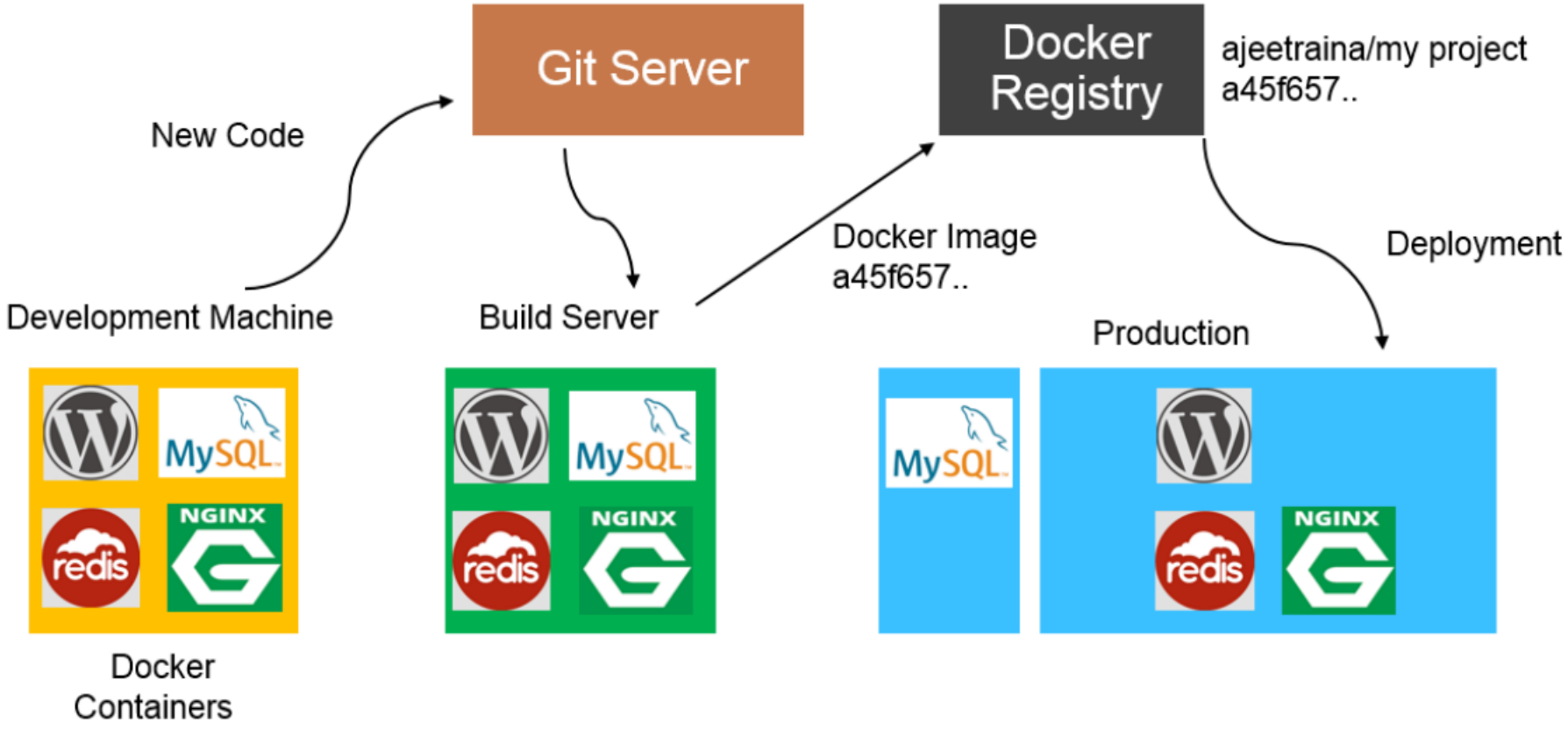
Development workflow (without Docker)



Development workflow (with Docker)



Development workflow (with Docker)



Installation

- The getting started guide on Docker has detailed instructions for setting up Docker on Mac, Linux and Windows

- On Mac/Windows - Two variants, Native and Toolbox

Lecture 02: Docker Containers

ehteshamzahoor — -bash — 80×24

```
>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
78445dd45222: Pull complete
Digest: sha256:c5515758d4c5e1e838e9cd307f6c6a0d620b5e07e6f927b07d05f6d12a1ac8d7
Status: Downloaded newer image for hello-world:latest

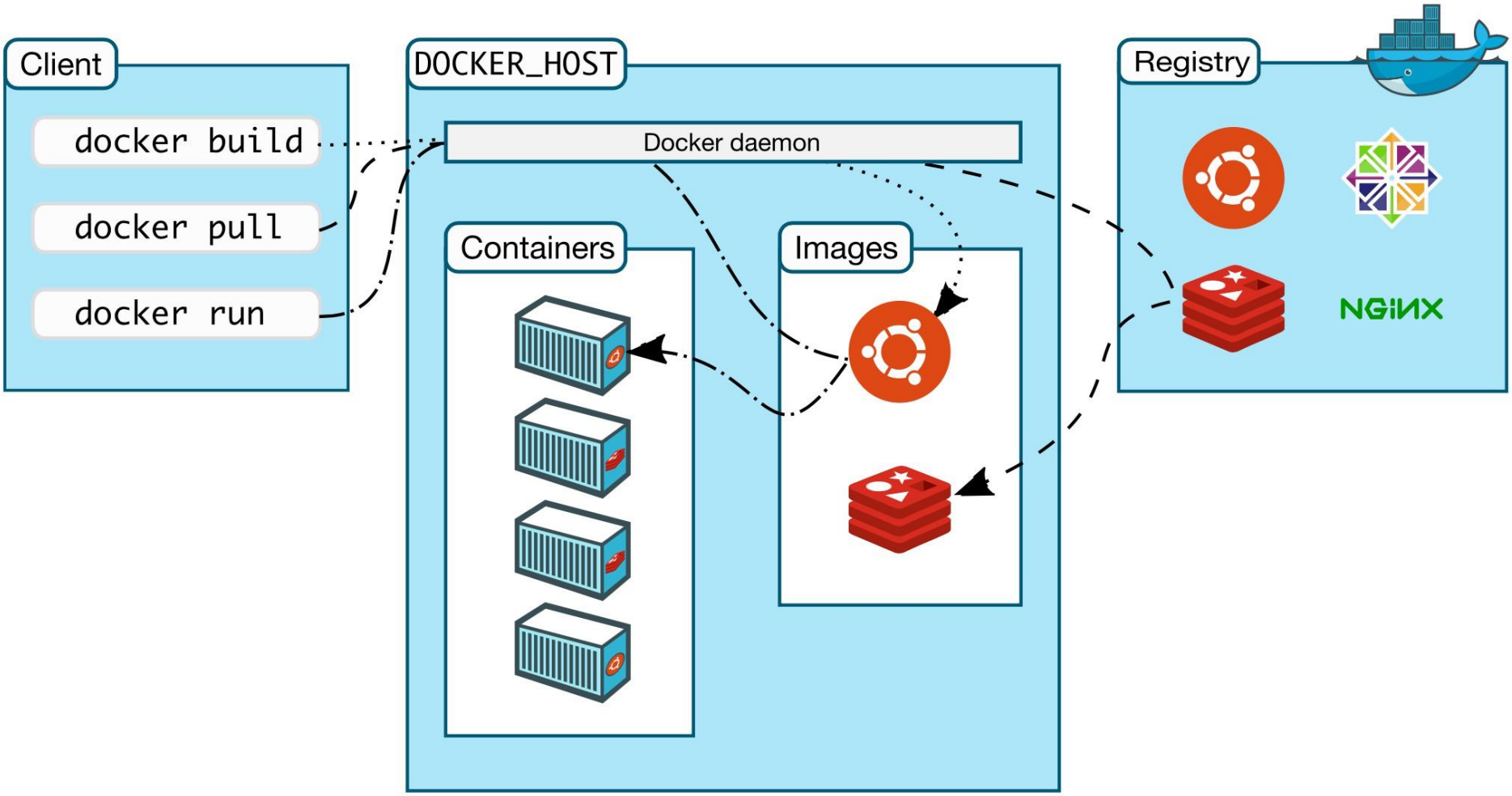
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

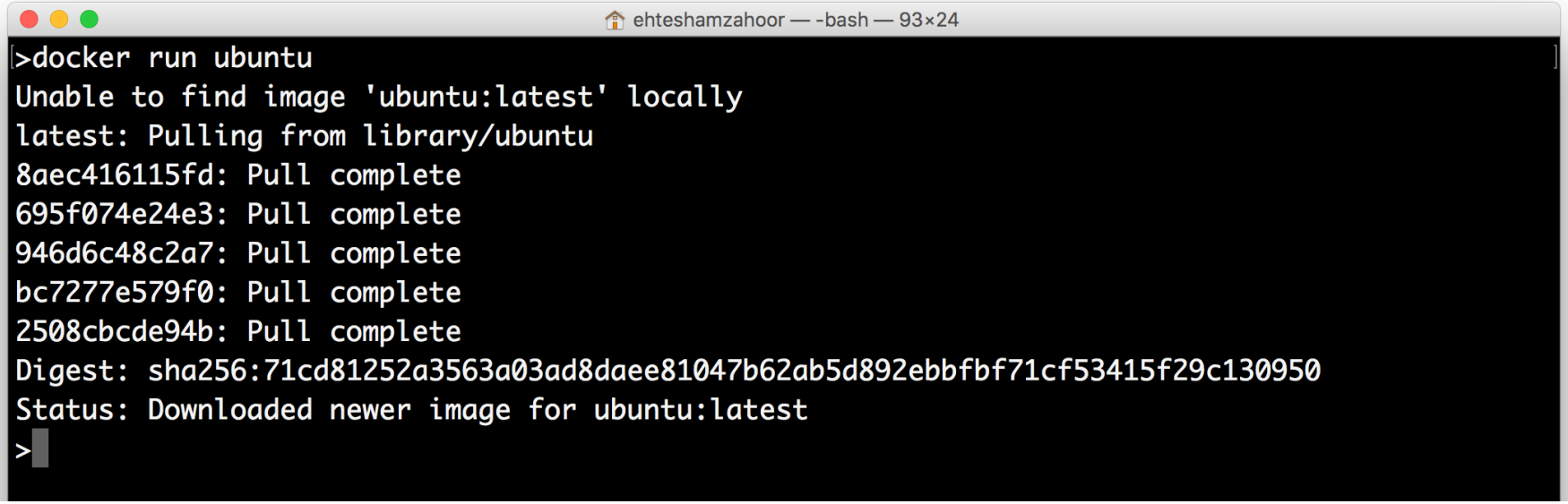
To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://cloud.docker.com/
```

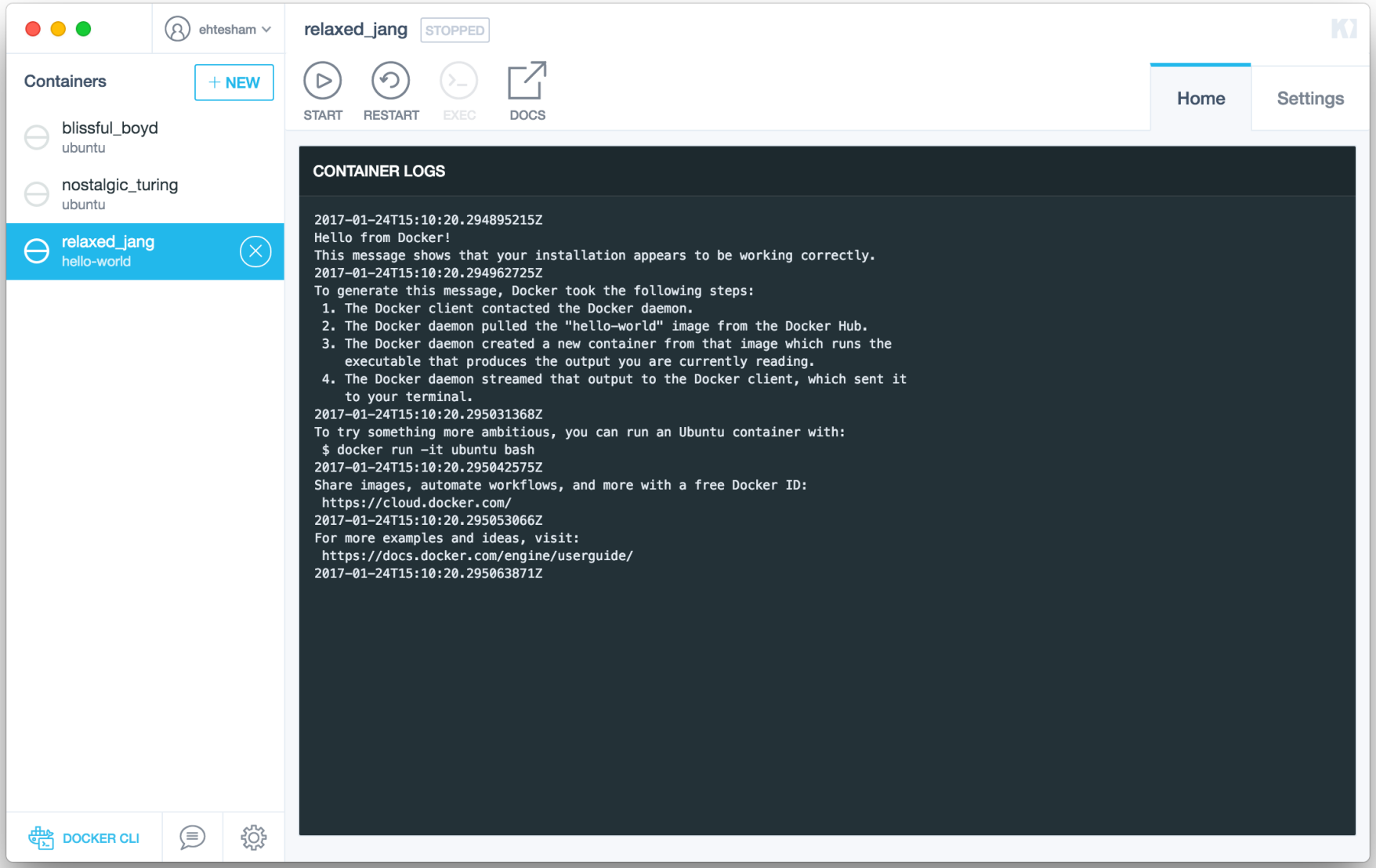
Docker's architecture



Lecture 02: Docker Containers



Lecture 02: Docker Containers



Deploying Webapp with docker

- Let us deploy a static one page Website with docker.

- We create a simple Web page in a directory, /Users/ehteshamzahoor/docker/staticweb, lets name it index.html

Deploying Webapp with docker

- In the same directory we create a file named Dockerfile with following contents

```
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

Deploying Webapp with docker

staticweb — -bash — 94×24

```
>pwd
/Users/ehteshamzahoor
>cd docker/staticweb/
>ls
Dockerfile      index.html
>docker build -t ehtesham/staticweb:latest .
Sending build context to Docker daemon 3.072 kB
Step 1/2 : FROM nginx:alpine
alpine: Pulling from library/nginx
b7f33cc0b48e: Pull complete
a362ec94dcab: Pull complete
825c3f8d9794: Pull complete
91e79a2321d5: Pull complete
Digest: sha256:30e3a72672e4c4f6a5909a69f76f3d8361bd6c00c2605d4bf5fa5298cc6467c2
Status: Downloaded newer image for nginx:alpine
 ---> f1fae62370b4
Step 2/2 : COPY . /usr/share/nginx/html
 ---> 31b25ed76b0f
Removing intermediate container 7ef6beec8bd5
Successfully built 31b25ed76b0f
>
```

Deploying Webapp with docker

staticweb — -bash — 94×24

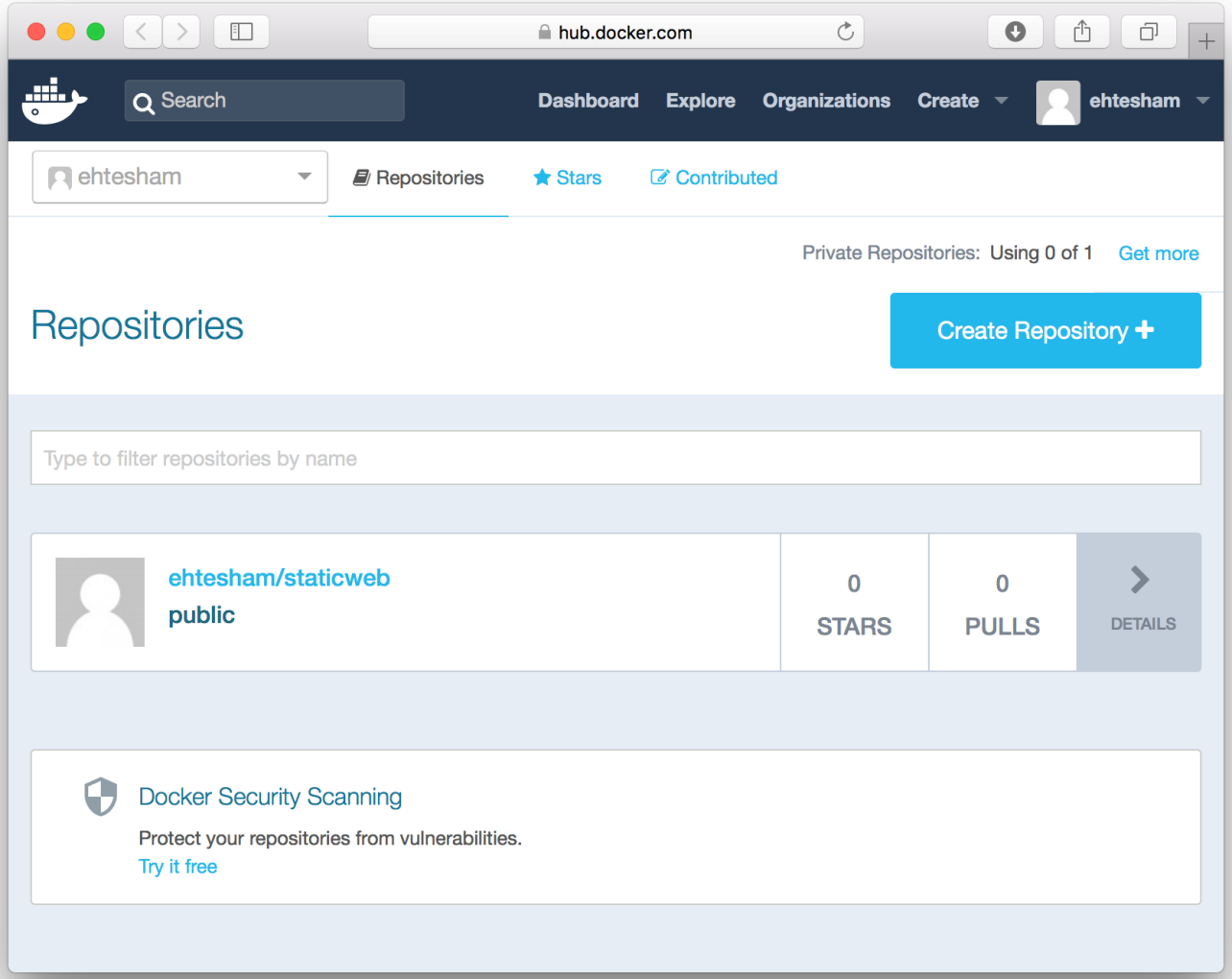
```
>docker images
REPOSITORY            TAG                 IMAGE ID            CREATED              SIZE
ehtesham/staticweb    latest              31b25ed76b0f        About a minute ago   54.3 MB
ubuntu                latest              f49eec89601e        3 days ago           129 MB
hello-world           latest              48b5124b2768        10 days ago          1.84 kB
nginx                 alpine              f1fae62370b4        3 weeks ago          54.3 MB
>
>
>
>docker run -p 1234:80 -d ehtesham/staticweb
4015e7f5384331369d0503b164dbd80cdd33be85ac555f765a28b90c2b77fc3d
>
>
```

Deploying Webapp with docker

staticweb — -bash — 95×27

```
>docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker I
D, head over to https://hub.docker.com to create one.
Username (ehtesham): ehtesham
Password:
Login Succeeded
>
>
>docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
ehtesham/staticweb  latest              31b25ed76b0f        10 hours ago        54.3 MB
ubuntu              latest              f49eec89601e        4 days ago          129 MB
hello-world         latest              48b5124b2768        11 days ago         1.84 kB
nginx               alpine              f1fae62370b4        4 weeks ago         54.3 MB
>
>
>docker push ehtesham/staticweb
The push refers to a repository [docker.io/ehtesham/staticweb]
c7f56ff8f739: Pushed
809c8c0dd73c: Mounted from library/nginx
e8d45b8ab3ca: Mounted from library/nginx
e8fa134cb7b8: Mounted from library/nginx
7cbcbac42c44: Mounted from library/nginx
latest: digest: sha256:474f82b3c654bd44dee3a4c68e891a8935490332f2c82420e98df969f7a50c79 size: 1
361
>
```

Deploying Webapp with docker



Deploying Webapp with docker

- We can use AWS BeansTalk to make our WebApp public !!

Running helloAOS - on Docker

```
#include <iostream>
using namespace std;


int main()
{
  cout << "Hello AOS \n";
  return 0;
}
```

Dockerfile

```
FROM gcc:4.9
RUN mkdir -p /app
COPY . /app
WORKDIR /app
RUN g++ helloaos.cpp -o hello
RUN chmod +x hello
CMD ["./hello"]
```